Abstract

The acquisition of programming skills is a notoriously difficult process characterized by high cognitive load, complex abstraction requirements, and significant dropout rates in higher education. Traditional pedagogical approaches often struggle to bridge the gap between abstract syntax and the strategic problem-solving required in software engineering. This paper presents a comprehensive gamified solution: a 2D top-down Role-Playing Game (RPG) inspired by classical adventure games like "The Legend of Zelda," specifically designed to reinforce existing knowledge. We detail the integration of a secure, Docker-based automated code judge within a narrative-driven environment. Key features include a virtual economy, a social "company" system for collaboration, and a curriculum focused on the C programming language to deepen low-level understanding. The paper evaluates the technical robustness of the prototype and proposes an empirical framework to validate the hypothesis that this immersive approach significantly enhances learning productivity, persistence, and student motivation.

2012 ACM Subject Classification Social and professional topics → Computer science education; Applied computing → Interactive learning environments

Keywords and phrases Gamification, C Programming, Role-Playing Games, Automated Grading, Docker

Digital Object Identifier magenta10.4230/OASIcs...

Category Regular Research Paper

Supplementary Material *Software*: magenta`https://github.com/example/c-zelda-prototype`

© Max Fritsche;
licensed under Creative Commons License CC-BY 4.0
OpenAccess Series in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

Gamification in Programming Education: A Zelda-inspired RPG Approach for Reinforcing Foundational Programming Skills

Max Fritsche
Duale Hochschule Sachsen - Staatliche Studienakademie Leipzig, Germany

1 Introduction

Programming education is facing a global crisis of engagement. As software becomes the backbone of modern infrastructure, the demand for developers grows, yet introductory computer science (CS1) courses report failure rates as high as 30-50%. The primary challenge lies not just in learning syntax, but in developing the strategic knowledge required to decompose complex problems and debug logic under pressure.

Current educational models often rely on "frontal" instruction followed by isolated, context-free exercises. These methods lack the immediate, iterative feedback loops that modern learners require to stay motivated. This paper argues that Gamification, the application of game design elements to non-game contexts, can provide a solution by fostering a "Flow" state where the challenge of coding is matched by an immersive narrative and immediate rewards.

1.1 Challenges in Programming Education

As noted by Singh (2022) and Medeiros et al. (2019), beginners struggle with three distinct levels of knowledge:

1. **Syntactic Knowledge:** The specific rules and grammar of the language (e.g., semicolons, brackets).
2. **Conceptual Knowledge:** Understanding abstract structures like loops, recursion, or pointers.
3. **Strategic Knowledge:** The ability to apply concepts to solve novel problems, plan algorithms, and systematically debug.

1.1.1 Cognitive and Subject-Specific Hurdles

Students frequently struggle with the step from conceptual knowledge to strategic application. While a student might understand what a "variable" is in isolation, they often fail to integrate multiple concepts into a functioning program. Syntax errors, which lead to frustrating "trial-and-error" loops, often overshadow the actual logic of the problem. Medeiros et al. (2019) emphasize that the high degree of abstraction, such as memory addresses and control structures, often lacks analogies in everyday life, making the initial learning curve exceptionally steep.

1.1.2 Behavioral and Emotional Factors

Beyond cognitive barriers, behavioral factors like self-efficacy and time management play a crucial role. Programming is often perceived as "inherently difficult," which leads to early demotivation. In large cohorts, individual support is limited, and students often feel abandoned when facing complex software setups or cryptic compiler errors. This lack of "psychological safety" inhibits exploration and experimentation.

© Max Fritsche;
licensed under Creative Commons License CC-BY 4.0
OpenAccess Series in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

1.2 Hypothesis of the Study

The central hypothesis of this work is: *"The gamification of programming education can be successfully and effectively implemented in the form of a Zelda-like RPG to increase learning productivity and motivation."* To validate this, we developed a prototype that focuses on the reinforcement of existing skills through practical, narrative-embedded exercises.

2 Theoretical Foundations

2.1 The Importance of Practical Exercise

Programming is a craft that requires active participation. According to the constructivist principle of "Learning by Doing," knowledge is constructed through experience. Iftikhar et al. (2022) state that "Learning to code is a skill that can only be developed by virtue of practice." Passive consumption of lectures or textbooks provides the vocabulary, but only the application in diverse scenarios builds actual competence.

2.2 Gamification as a Didactic Method

Gamification refers to the use of game-design elements in non-game contexts to increase user engagement (Smiderle et al., 2020). Unlike "Games for Learning," where the game is the primary focus, gamification adds a layer of motivation to the existing learning content.

2.2.1 Behaviorist Reinforcement

The core loop of a gamified system is inherently behaviorist. Based on B.F. Skinner's theories (1958), behavior is shaped by its consequences. Immediate feedback, such as points or visual rewards, serves as a positive reinforcer. Our system provides this through an Online Code Judge that evaluates code in seconds, allowing students to learn from mistakes instantly rather than waiting days for manual grading.

2.2.2 Flow Theory and Scaffolding

Optimal learning occurs in the "Flow Zone," a state between boredom and anxiety. Mihaly Csíkszentmihályi describes this as a state of total immersion. In our RPG, "Scaffolding" is used to provide a ladder of increasing difficulty. Early quests are simple, building the student's confidence, while later quests challenge their strategic thinking.

2.2.3 Psychological Motivation Frameworks

We apply two major frameworks to ensure deep engagement:

- **Self-Determination Theory (SDT):** Developed by Deci and Ryan (2016), this theory identifies three basic needs: *Autonomy* (choice of quests), *Competence* (mastering C code), and *Relatedness* (social interaction in companies).
- **ARCS Model:** Developed by Keller, this model focuses on Attention (narrative), Relevance (real-world code), Confidence (clear goals), and Satisfaction (rewards).

2.3 Gamification vs. Traditional Methods

Traditional methods often suffer from high latency in feedback and a "punitive" error culture. Errors are seen as failure (grade deduction) rather than a learning opportunity. In a gamified

RPG context, a "Game Over" or a failed test case is an invitation to retry. Studies by Lampropoulos et al. (2024) show that students in gamified courses achieved 14% higher pass rates and 17% higher average grades compared to traditional groups.

3 State of the Art: Analysis of Existing Platforms

A review of current tools reveals specific strengths and weaknesses in the context of higher education.

3.1 Block-based Environments

Scratch is the industry standard for k-12. It avoids syntax errors through drag-and-drop blocks. While effective for teaching logic, it is often viewed as overly simplistic by university students. Furthermore, it abstracts away critical low-level concepts like memory management or data types, which are necessary for professional engineering.

3.2 Puzzle and Logic Games

Games like *Lightbot* teach algorithmic thinking through spatial puzzles. They are highly engaging but stay too abstract. They lack the transition to text-based coding and authentic development workflows.

3.3 Code Adventure Games

CodeCombat and *CheckiO* use real code (Python/JS) to control characters in a story. While successful, these platforms often situate programming tasks primarily within a game environment, which may hinder the transfer of skills to local development environments. Our approach focuses on C to provide a "harder" academic challenge suitable for computer science majors.

3.4 Online Code Judges

LeetCode and *HackerRank* are high-performance tools for competitive programming. They offer immediate feedback but lack narrative context. For many beginners, these platforms feel like "dry" testing environments rather than motivating learning spaces.

4 Conceptual Design of the Learning Game

4.1 Target Group and Learning Goals

The game targets second- or third-semester students who have already completed an introductory course. The goals are:

- **Strategic Thinking:** Solving complex problems through algorithm design.
- **Debugging Competence:** Systematic error identification and correction.
- **Low-Level Understanding:** Mastery of pointers, structs, and manual memory management in C.

4.2 The Zelda Metaphor and Movement

We chose a 2D top-down pixel-art style, inspired by *The Legend of Zelda*. This aesthetic is nostalgic, accessible, and reduces cognitive load compared to complex 3D environments.

- **Exploration:** Players move through a spatial world, making the learning path tangible.
- **Interactions:** NPCs act as mentors or "Quest Givers."
- **Contextualization:** Coding tasks are presented as "rebuilding broken bridges" or "deciphering ancient scrolls."

4.3 Socio-Economic Ecosystem

The game features a meta-layer to ensure long-term persistence:

- **Virtual Economy:** Students earn "Credits" for solving quests.
- **Company System:** Students can form "Companies" (teams) to pool resources. This mimics real-world development teams and encourages peer-instruction.
- **Properties:** Companies can buy "Offices" or "Labs" in the game world, providing a sense of ownership and social status.
- **Social Feed:** A real-time activity feed shows achievements of friends, fostering a healthy sense of community.

5 Technical Implementation

5.1 System Architecture

The prototype utilizes a distributed, multi-tier architecture:

1. **Unity Client:** A standalone 2D application handling world rendering, movement, and the integrated code editor.
2. **Node.js/Express Backend:** Manages the business logic, quest state, and economy.
3. **Supabase/PostgreSQL:** Provides persistent storage with Row-Level Security (RLS) to ensure data privacy.
4. **Docker Runner Service:** An isolated environment for safe code execution.

5.2 Backend and Security Design

The Node.js server acts as the central coordinator. For security, we utilize JSON Web Tokens (JWT) for authentication. Every interaction, from buying an item to starting a quest, is validated against the user's role and state. The database schema includes specialized tables for `users`, `quests`, `transactions`, `companies`, and `feed_items`.

5.3 The Dockerized Code Runner

Executing arbitrary user code (especially in C) is a massive security risk. We implemented a "Sandbox" model:

- **Isolation:** Each run occurs in a fresh Docker container with no network access.
- **Resource Limits:** Containers are capped at 256MB RAM and 5 seconds of CPU time.
- **Redis Queue:** To handle spikes in submissions, jobs are pushed to a Redis queue and processed by a pool of "pre-warmed" containers.
- **Feedback Loop:** The runner captures `stdout`, `stderr`, and exit codes, sending them back to Unity via Server-Sent Events (SSE).

5.4 Unity Movement and Animation System

Technical excellence in the frontend is key for immersion.

- **Physics:** A `Rigidbody2D` based movement system ensures smooth collisions.
- **Animations:** A state-machine-based Animation Controller handles transitions between "Idle" and "Walk" for four directions.
- **Input:** We implemented the new Unity Input System to support both Keyboard and Gamepads.

6 Technical Validation

6.1 Code Judge Reliability

The judge was tested with several scenarios:

- **Infinite Loops:** A program with `while(1);` was correctly terminated by the Docker engine after 5 seconds.
- **Memory Abuse:** A program attempting to allocate 2GB of RAM was killed by the OOM-killer.
- **Syntax Parsing:** Compiler errors from `gcc` were correctly passed through to the game terminal.

6.2 Movement and Interaction Tests

We validated the interaction triggers. An NPC only opens the "Quest Menu" when the player is within a specific 2D trigger zone and presses the interaction key. This prevents overlapping UI elements and ensures a clean user experience.

7 Proposed Empirical Evaluation

To confirm the pedagogical impact, we propose a longitudinal study design.

7.1 Study Design

A cohort of 60 bachelor students will be split into an *Experimental Group* (RPG) and a *Control Group* (Traditional).

- **Period:** 6 weeks of intervention.
- **Topics:** Identical for both groups (Arrays, Pointers, Structs, Linked Lists).
- **Metries:** Pre- and Post-tests for strategic knowledge, time-on-task, and completion rates.

7.2 Didactic Importance of Quests

The quest structure provides narrative framing. Instead of "Write a function for Bubble Sort," the quest is "Organize the Kingdom's Library." This contextualization makes the problem-solving process feel meaningful. Qualitative feedback from previous studies (Flores et al., 2023) suggests that students describe such courses as "gripping" and "fesselnd."

8 Conclusion and Future Work

8.1 Summary of Results

The "C-Zelda" prototype demonstrates that gamification can be successfully integrated into the rigorous context of university programming education. Technically, we have shown that Docker-based code execution is safe and responsive. Pedagogically, the combination of narrative exploration and immediate feedback addresses the primary pain points of beginners: demotivation and the struggle with abstraction.

8.2 Outlook and School Transfer

For future development, we envision:

- **Multi-Language Support:** Expanding the runner to support Python and Rust.
- **Collaborative Dungeons:** Multi-player puzzles where code from two players must be synchronized to succeed.
- **School Integration:** Adapting the narrative for the German school system (Oberstufe) to promote "Informatische Grundbildung" in a playful yet serious way.

References

1. Medeiros, R. P., et al. (2019). A Systematic Literature Review on Teaching and Learning Introductory Programming in Higher Education. IEEE Transactions on Education.
2. Singh, S. (2022). Identifying Learning Challenges faced by Novice Computer Programming Students: An Action Research Approach.
3. Iftikhar, S., et al. (2022). Practice Promotes Learning: Analyzing Students' Acceptance of a Learning-by-Doing Online Programming Learning Tool.
4. Lampropoulos, G., & Sidiropoulos, A. (2024). Impact of Gamification on Students' Learning Outcomes and Academic Performance: A Longitudinal Study. Education Sciences.
5. Skinner, B. F. (1958). Teaching machines. Science.
6. Csíkszentmihályi, M. (1990). Flow: The Psychology of Optimal Experience.
7. Zhan, Z., et al. (2022). The effectiveness of gamification in programming education: Evidence from a meta-analysis. Computers and Education: AI.
8. Deterding, S. (2016). Contextual Autonomy Support in Video Game Play: A Grounded Theory.